
Reinforcement Learning Assignment 1

Tabular Reinforcement Learning

Rahul Ray¹ Chi-Yuan Kiong¹ Jian Song¹

Abstract

Tabular reinforcement learning comprises a family of fundamental algorithms that reveal deep insights into how agents learn from interaction with an environment. We study several such algorithms in the Stochastic Windy Gridworld, a 10×7 grid where probabilistic wind makes navigation non-trivial. Starting from Dynamic Programming as an optimal baseline (converging in 17 iterations to a return of 83.68), we evaluate model-free methods in two ways, exploration strategy (ϵ -greedy vs. Boltzmann) and backup depth (1-step Q-learning, SARSA, n -step Q-learning, and Monte Carlo). All results are averaged over 200 repetitions with 95% confidence intervals. No model-free method reaches the DP optimum, highlighting the fundamental cost of operating without an environmental model.

1. Introduction

Reinforcement learning (RL) enables agents to learn optimal behavior through interaction with an environment without supervision. A core challenge is to balance exploration of unknown states with exploitation of current knowledge. Algorithmic choices — such as the use of environment model, backup targets, exploration strategies — affect learning efficiency, stability, and optimality.

In this study, we analyze different model-free methods, exploration strategies, and backup depths on the Stochastic Windy Gridworld, a 10×7 navigation task where probabilistic wind makes transitions stochastic. Our goal is to understand the trade-offs inherent in RL and identify conditions under which certain approaches succeed or fail. To ensure reliable evaluation, results are averaged over multiple repetitions (200 repetitions with 95% confidence intervals).

^{*}Equal contribution ¹Faculty of Science, Leiden University, Leiden, the Netherlands. Correspondence to: Rahul Ray <s4875710@vuw.leidenuniv.nl>, Chi-Yuan Kiong <s4623304@vuw.leidenuniv.nl>, Jian Song <s4870832@vuw.leidenuniv.nl>.

2. Theory

In reinforcement learning, an agent interacts with an environment formalized as a Markov Decision Process (MDP). At each time step t , the agent in state s_t selects an action a_t , receives a reward r_{t+1} , and transitions to a new state s_{t+1} . The goal is to derive an optimal policy $\pi^*(a|s)$ that maximizes the expected cumulative return over time. (Sutton & Barto, 2018)

2.1. Dynamic Programming

Dynamic Programming (DP) assumes full access to the environment model, including transition probabilities $p(s'|s, a)$ and reward function $r(s, a, s')$. It computes exact action-values by sweeping over all state-action pairs iteratively, using Q-value iteration (Algorithm 1)-

$$Q(s, a) \leftarrow \sum_{s', r} p(s', r|s, a) [r + \gamma \max_{a'} Q(s', a')] \quad (1)$$

Here, $Q(s, a)$ is the expected return of taking action a in state s , and $\gamma \in [0, 1]$ is the discount factor. DP provides exact solutions but requires a complete model and can become infeasible in large state spaces due to the curse of dimensionality. (Sutton & Barto, 2018)

2.2. Exploration and Q-learning Update

When the environment model is unavailable, the agent must balance exploration and exploitation to learn effectively. Common strategies include-

$$\epsilon\text{-greedy: } \pi(a|s) = \begin{cases} 1 - \epsilon \cdot \frac{|\mathcal{A}|-1}{|\mathcal{A}|}, & a = \arg \max_b \hat{Q}(s, b) \\ \epsilon/|\mathcal{A}|, & \text{otherwise} \end{cases} \quad (2)$$

$$\text{Boltzmann: } \pi(a|s) = \frac{e^{\hat{Q}(s, a)/\tau}}{\sum_b e^{\hat{Q}(s, b)/\tau}} \quad (3)$$

Here, $\hat{Q}(s, a)$ is the estimated action-value, and τ controls the randomness in action selection. Action-values are updated incrementally using 1-step Q-learning:

$$\hat{Q}(s_t, a_t) \leftarrow \hat{Q}(s_t, a_t) + \alpha [r_{t+1} + \gamma \max_{a'} \hat{Q}(s_{t+1}, a') - \hat{Q}(s_t, a_t)] \quad (4)$$

where α is the learning rate. This update moves the current estimate toward the observed reward plus the discounted

estimate of the best next action, allowing the agent to learn from experience even without a model.

2.3. Temporal Difference (TD) Learning

Q-learning and SARSA are both instances of TD learning (Sutton & Barto, 2018), combining bootstrapping with observed rewards. Q-learning (off-policy) assumes that the agent will act optimally in the future, updating toward the maximum estimated next action-value (Algorithm 2). SARSA (on-policy) updates according to the action actually taken next, which can produce more stable learning (Algorithm 3):

$$\hat{Q}(s_t, a_t) \leftarrow \hat{Q}(s_t, a_t) + \alpha [r_{t+1} + \gamma \hat{Q}(s_{t+1}, a_{t+1}) - \hat{Q}(s_t, a_t)] \quad (5)$$

Here, the TD error (the term in brackets) represents the difference between the observed return and the current estimate, guiding the update.

2.4. Backup Depth and Targets

TD updates can be generalized to n -step targets, passing rewards over multiple transitions (Algorithm 4):

$$G_{t:t+n} = \sum_{i=0}^{n-1} \gamma^i r_{t+i+1} + \gamma^n \max_a \hat{Q}(s_{t+n}, a) \quad (6)$$

At the extreme, Monte Carlo (MC) omits bootstrapping entirely, summing all rewards to episode end (Algorithm 5):

$$G_t = \sum_{i=0}^{\infty} \gamma^i r_{t+i+1} \quad (7)$$

$$\hat{Q}(s_t, a_t) \leftarrow \hat{Q}(s_t, a_t) + \alpha [G_t - \hat{Q}(s_t, a_t)] \quad (8)$$

This n -step target bridges 1-step TD and full Monte Carlo returns. Smaller n relies more on bootstrapping (lower variance, potentially higher bias), while larger n accumulates more actual rewards (lower bias, higher variance). By choosing n , one can trade off between stability and responsiveness in learning.

3. Experiment

3.1. Setup

All experiments are conducted in the Stochastic Windy Grid-world, a 10×7 grid where the agent navigates from S(0,3) to G(7,3) against a stochastic upward wind (90% probability), with a step reward of -1 and terminal reward of $+100$. Episodes are capped at 100 steps. To account for variability, we wrote a custom outer loop repeating `Experiment.py` 10 times over 20 inner repetitions, yielding 200 total runs per setting. Results showed 95% confidence intervals via t -distribution and are smoothed with a window of 11. All experiments use $\gamma = 1.0$.

3.2. Results

3.2.1. DYNAMIC PROGRAMMING

We ran Q-value iteration with $\gamma = 1.0$ and $\eta = 0.001$, converging in 17 iterations. Figure 4 shows the progression of value estimates.

The optimal start value is $V^*(s = 3) = 83.68$, reflecting the expected cumulative reward from the start, given a step reward of -1 and terminal reward of $+100$. This corresponds to an expected path length of $E[T] \approx 17$ steps, yielding an average reward per step of 4.83 (computed as $V^*(s = 3)$ divided by $E[T]$). Terminal states are handled by setting $Q(\text{goal}, a) = 0$, meaning that no future reward is obtained after reaching the goal, which ensures convergence. Alternatively, terminal states could be excluded from the update sweep or implemented with self-loop transitions and zero reward. Moving the goal to (6, 2) drops V^* to 64.88 and increases convergence from 17 to 132 iterations. Despite being closer, the new goal sits in a high-wind column, demonstrating that DP captures stochasticity and not just spatial distance.

3.2.2. EXPLORATION STRATEGIES

We compared ϵ -greedy ($\epsilon \in \{0.03, 0.1, 0.3\}$) and softmax ($\tau \in \{0.01, 0.1, 1.0\}$) using Q-learning with $\alpha = 0.1$, averaged over 200 runs.

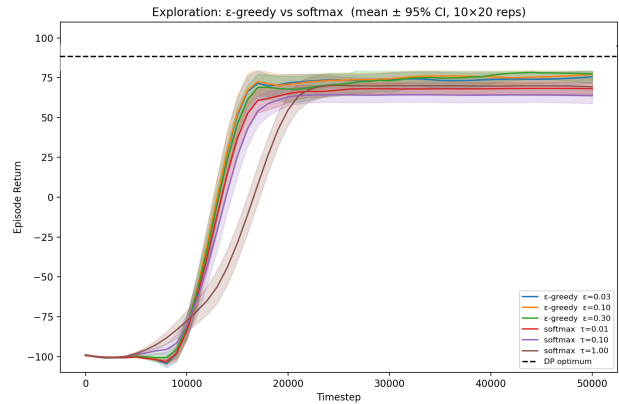


Figure 1. Learning curves comparing ϵ greedy and softmax exploration strategies using Q-learning with $\alpha = 0.1$, averaged over 200 runs with 95% confidence intervals. Among ϵ -greedy variants, $\epsilon = 0.3$ achieves the highest asymptotic return (80), demonstrating that moderate exploration outperforms both overly conservative ($\epsilon = 0.03$) and overly greedy settings. Boltzmann exploration consistently underperforms ϵ -greedy across all tested temperature values. No model-free method reaches the DP optimum of 83.68, reflecting the fundamental cost of learning without environment model access.

As shown in Figure 1, all methods eventually converge,

with ϵ -greedy achieving the highest asymptotic value while Boltzmann performs worse. No model-free method reaches the DP optimum of 83.68. Compared to the single-goal setup, the previously optimal exploration parameter ($\epsilon = 0.3$) no longer achieves high returns; all values of ϵ and τ fail to overcome the local reward trap. This suggests that stronger or adaptive exploration strategies may be required, as fixed exploration parameters are insufficient to reliably discover the distant high-reward goal.

Introducing a secondary goal at $(3, 2)$ with $R = +5$ creates a local optimum trap (Figure 5), where agents exploit the nearby reward instead of the $+100$ global goal. This illustrates a critical failure in the exploration-exploitation trade-off: early visits to the secondary goal dominate the action-value estimates, causing the agent to prematurely exploit the local reward before discovering the more distant $+100$ goal. Increasing ϵ or adjusting τ does not resolve this issue, as the local optimum heavily biases early learning trajectories.

3.2.3. BACK-UP: ON-POLICY VERSUS OFF-POLICY

We compared Q-learning and SARSA at $\alpha \in \{0.03, 0.1, 0.3\}$ with $\epsilon = 0.1$, averaged over 200 runs.

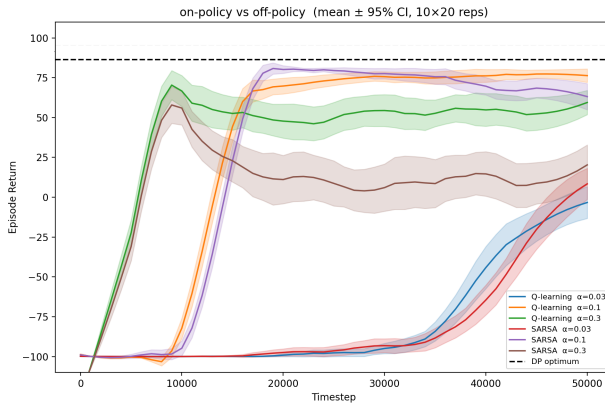


Figure 2. Comparison of Q-learning (off-policy) versus SARSA (on-policy) across learning rates $\alpha \in \{0.03, 0.1, 0.3\}$, averaged over 200 runs with 95% confidence intervals. Q-learning with $\alpha = 0.1$ achieves the best performance. At $\alpha = 0.3$, both methods exhibit instability, with Q-learning showing high variance oscillations. At $\alpha = 0.03$, convergence is too slow within the 50,000 timesteps.

As is shown in 2, Q-learning outperforms SARSA at moderate learning rates due to its off-policy updates, which learn the optimal policy independently of exploratory actions. Both methods converge slowly at very low α and become unstable at high α . We select Q-learning with $\alpha = 0.1$ as the preferred method for this task.

3.2.4. BACK-UP: DEPTH OF TARGET

We compared n -step Q-learning ($n \in \{1, 3, 10\}$) against Monte Carlo using $\epsilon = 0.1$, $\alpha = 0.1$, averaged over 200 runs.

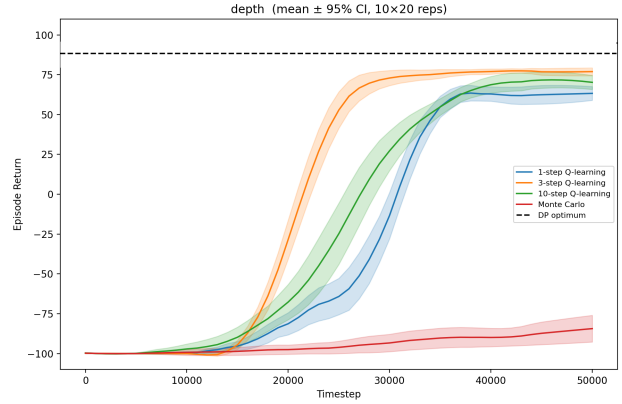


Figure 3. Learning curves for n -step Q-learning with varying backup depths ($n \in \{1, 3, 10\}$) and Monte Carlo, using $\epsilon = 0.05$ and $\alpha = 0.1$, averaged over 200 runs with 95% confidence intervals. The 3-step method achieves the best balance between bias and variance, reaching ≈ 76 and converging fastest. Monte Carlo performs worst due to high variance from accumulating stochastic wind effects over entire episodes. At $n = 10$, longer traces accumulate excessive variance, degrading final performance compared to $n = 3$.

As is shown in 3, Monte Carlo performs worst due to high variance from long stochastic episodes. Among n -step methods, $n = 3$ achieves the best performance (≈ 76), balancing bias and variance, while 1-step and 10-step reach lower asymptotic values. We select $n = 3$ as the preferred backup depth for this environment.

4. Discussion

4.1. DP versus RL

DP finds the optimal policy in 17 iterations with full model access, sweeping through the entire state space systematically without environment interaction (Sutton & Barto, 2018). The weakness is that a complete model is rarely available in practice, and DP suffers from the curse of dimensionality. Model-free RL sidesteps the model requirement entirely but pays with higher sample complexity and no optimality guarantee, as our results clearly show.

4.2. Exploration

We prefer ϵ -greedy for its robustness to Q-value scale. Boltzmann is sensitive to Q-value magnitudes, which span -100 to $+100$ in this environment, making temperature tuning difficult. A more principled alternative would be UCB (Auer

et al., 2002), which explicitly tracks visitation counts, or intrinsic motivation methods that reward visiting novel states, both would help in the multi-goal case where local optima trapped all agents.

4.3. Effect of Initialising Rewards at 0

The -1 step reward with $\hat{Q} = 0$ initialisation creates implicit optimistic exploration — unvisited states look better than visited ones, encouraging early exploration (Sutton & Barto, 2018). Setting step reward to 0 removes this incentive entirely. All state-action pairs remain at 0 until the goal is reached, meaning the agent has no signal to distinguish good from bad actions, slowing convergence considerably.

4.4. Discount Factor with Zero Step Reward

With zero step rewards, $\gamma = 1.0$ makes all path lengths equivalent — every path yields exactly $+100$ regardless of length, so the agent has no reason to prefer shorter paths. Only $\gamma < 1$ (e.g., 0.99) incentivises shortest paths, since a path of length k yields $0.99^k \cdot 100$, which is strictly higher for smaller k (Sutton & Barto, 2018).

4.5. On-policy versus Off-policy

Q-learning bootstraps with $\max_{a'} \hat{Q}(s', a')$, learning the optimal policy value regardless of actions taken (Watkins & Dayan, 1992). SARSA bootstraps with $\hat{Q}(s', a')$ for the action actually selected, learning the behavioural policy value including exploration noise (Rummery & Niranjan, 1994). In our stochastic environment, SARSA's on-policy target accumulated noise from exploratory actions, slowing convergence. n -step Q-learning is hybrid — the first n rewards are on-policy, but the final bootstrap is off-policy. SARSA might be preferred in highly stochastic or risky environments, where exploratory actions could lead to large negative rewards, as its on-policy nature accounts for the actual policy being followed.

4.6. Backup Depth and Bias-Variance Trade-off

1-step methods have low variance but high bias, as they bootstrap heavily from potentially inaccurate early estimates. MC has zero bias but suffers from crippling variance in stochastic environments (Sutton & Barto, 2018). In our experiments, $n = 3$ balanced reward propagation speed against variance accumulation optimally. At $n=10$, longer traces accumulated more wind induced variance, leading to less stable final performance. Comparing standard Q-learning with 1-step Q-learning from the n -step framework reveals similar performance but different learning dynamics. The key implementation difference is that standard Q-learning updates after every transition, while n -step Q-learning collects entire episodes before updating. This batch

structure means 1-step n -step Q-learning makes fewer but larger update passes, potentially affecting early learning speed despite identical backup targets when $n = 1$.

4.7. Curse of Dimensionality

Tabular methods store a separate entry for every state-action pair. Our 280-entry Q-table is trivial, but problems with continuous states or high-dimensional observations grow exponentially (Bellman, 1957). Function approximation addresses this by generalising across similar states through shared parameters. DQN (Mnih et al., 2015) demonstrated that combining Q-learning with neural networks scales to high-dimensional inputs like raw pixels.

5. Conclusion

We studied tabular reinforcement learning on the Stochastic Windy Gridworld, comparing Dynamic Programming against model-free methods across three dimensions- exploration strategy, backup type, and backup depth.

Dynamic Programming converged to the optimal policy in just 17 iterations with $V^*(s = 3) = 83.68$, demonstrating the power of full model access. No model-free method reached this optimum, highlighting the fundamental cost of learning without a model. Among exploration strategies, ϵ -greedy outperformed softmax and proved robust across parameter settings, with $\epsilon = 0.3$ achieving the best asymptotic return at ≈ 80 . For backup type, Q-learning dominated SARSA at moderate learning rates ($\alpha = 0.1$), though both struggled at the extremes, too high caused instability, too low prevented convergence within the budget. Finally, 3-step Q-learning emerged as the best backup depth, balancing reward propagation speed against variance accumulation, while Monte Carlo failed entirely due to the high variance of full-episode returns in a stochastic environment.

These results reinforce core RL principles - model access enables optimality, off-policy learning accelerates convergence when exploration noise is present, and intermediate backup depths offer the best bias-variance trade-off. Future work could explore adaptive methods such as eligibility traces or curiosity-driven exploration to further close the gap with the DP optimum.

References

- Auer, P., Cesa-Bianchi, N., and Fischer, P. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2–3), 2002.
- Bellman, R. *Dynamic Programming*. Princeton University Press, Princeton, NJ, 1957.
- Mnih, V., Kavukcuoglu, K., Silver, D., et al. Human-level control through deep reinforcement learning. *Nature*, 518 (7540), 2015.
- Rummery, G. A. and Niranjan, M. On-line Q-learning using connectionist systems. Technical Report CUED/F-INFENG/TR 166, Cambridge University Engineering Department, 1994.
- Sutton, R. S. and Barto, A. G. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 2nd edition, 2018.
- Watkins, C. J. C. H. and Dayan, P. Q-learning. *Machine Learning*, 8(3–4), 1992.

A. Algorithms

Algorithm 1 Tabular Q-value Iteration (Dynamic Programming)

Input: Threshold $\eta \in \mathbb{R}^+$, discount γ

Result: Optimal value function $Q^*(s, a)$ and policy $\pi^*(s)$

```

1  $\hat{Q}(s, a) \leftarrow 0 \quad \forall s \in \mathcal{S}, a \in \mathcal{A}$  repeat
2    $\Delta \leftarrow 0$  for each  $s \in \mathcal{S}$  do
3     for each  $a \in \mathcal{A}$  do
4        $x \leftarrow \hat{Q}(s, a)$  // store current
        estimate
5        $\hat{Q}(s, a) \leftarrow \sum_{s'} p(s'|s, a)[r(s, a, s') +$ 
         $\gamma \max_{a'} \hat{Q}(s', a')]$   $\Delta \leftarrow \max(\Delta, |x -$ 
         $\hat{Q}(s, a)|)$  // track max change
6 until  $\Delta < \eta$ 
7 return  $Q^*(s, a) = \hat{Q}(s, a), \quad \pi^*(s) =$ 
         $\arg \max_a Q^*(s, a) \quad \forall s \in \mathcal{S}$ 
```

Algorithm 2 Tabular Q-learning

Input: Learning rate $\alpha \in (0, 1]$, discount γ , exploration parameter ϵ , total budget

```

8  $\hat{Q}(s, a) \leftarrow 0 \quad \forall s \in \mathcal{S}, a \in \mathcal{A}$   $s \sim p_0(s)$  // sample
   initial state
9 while budget do
10    $a \sim \pi(a|s)$  // e.g.  $\epsilon$ -greedy or softmax
11    $r, s' \sim p(r, s'|s, a)$  // simulate
       environment
12    $\hat{Q}(s, a) += \alpha[r + \gamma \max_{a'} \hat{Q}(s', a') - \hat{Q}(s, a)]$  if  $s'$ 
       is terminal then
13      $s \sim p_0(s)$  // reset environment
14   else
15      $s \leftarrow s'$ 
16 return  $\hat{Q}(s, a)$ 
```

Algorithm 3 Tabular SARSA

Input: Learning rate $\alpha \in (0, 1]$, discount γ , exploration parameter ϵ , total budget

```

17  $\hat{Q}(s, a) \leftarrow 0 \quad \forall s \in \mathcal{S}, a \in \mathcal{A} \quad s \sim p_0(s) \quad // \text{ sample initial state}$ 
18  $a \sim \pi(a|s) \quad // \text{ sample first action}$ 
19 while budget do
20    $r, s' \sim p(r, s'|s, a) \quad // \text{ simulate environment}$ 
21    $a' \sim \pi(a'|s') \quad // \text{ sample next action from policy}$ 
22    $\hat{Q}(s, a) += \alpha[r + \gamma\hat{Q}(s', a') - \hat{Q}(s, a)]$  if  $s'$  is terminal then
23      $s \sim p_0(s) \quad a \sim \pi(a|s)$ 
24   else
25      $s \leftarrow s' \quad a \leftarrow a'$ 
26 return  $\hat{Q}(s, a)$ 
    
```

Algorithm 5 Tabular Monte Carlo RL

Input: Learning rate $\alpha \in (0, 1]$, discount γ , exploration parameter ϵ , max episode length T

```

40  $\hat{Q}(s, a) \leftarrow 0 \quad \forall s \in \mathcal{S}, a \in \mathcal{A}$  while budget do
41    $s_0 \sim p_0(s)$  for  $t = 0, \dots, T - 1$  do
42      $a_t \sim \pi(a|s_t) \quad r_t, s_{t+1} \sim p(r, s'|s_t, a_t)$  if  $s_{t+1}$  is terminal then
43       break
44    $G_{t+1} \leftarrow 0 \quad // \text{ start reward summation from 0}$ 
45   for  $i = t, \dots, 0$  do
46      $G_i \leftarrow r_i + \gamma \cdot G_{i+1} \quad // \text{ MC target at each step}$ 
47      $\hat{Q}(s_i, a_i) += \alpha[G_i - \hat{Q}(s_i, a_i)]$ 
48 return  $\hat{Q}(s, a)$ 
    
```

Algorithm 4 Tabular n -step Q-learning

Input: Learning rate $\alpha \in (0, 1]$, discount γ , exploration parameter ϵ , max episode length T , depth n

```

27  $\hat{Q}(s, a) \leftarrow 0 \quad \forall s \in \mathcal{S}, a \in \mathcal{A}$  while budget do
28    $s_0 \sim p_0(s)$  for  $t = 0, \dots, T - 1$  do
29      $a_t \sim \pi(a|s_t) \quad r_t, s_{t+1} \sim p(r, s'|s_t, a_t)$  if  $s_{t+1}$  is terminal then
30       break
31    $T_{ep} \leftarrow t + 1 \quad // \text{ episode length}$ 
32   for  $t = 0, \dots, T_{ep} - 1$  do
33      $m \leftarrow \min(n, T_{ep} - t) \quad // \text{ rewards available to sum}$ 
34     if  $s_{t+m}$  is terminal then
35        $G_t \leftarrow \sum_{i=0}^{m-1} \gamma^i r_{t+i} \quad // \text{ no bootstrap}$ 
36     else
37        $G_t \leftarrow \sum_{i=0}^{m-1} \gamma^i r_{t+i} + \gamma^m \max_a \hat{Q}(s_{t+m}, a)$ 
38      $\hat{Q}(s_t, a_t) += \alpha[G_t - \hat{Q}(s_t, a_t)]$ 
39 return  $\hat{Q}(s, a)$ 
    
```

B. Progression of Q-Value iteration.

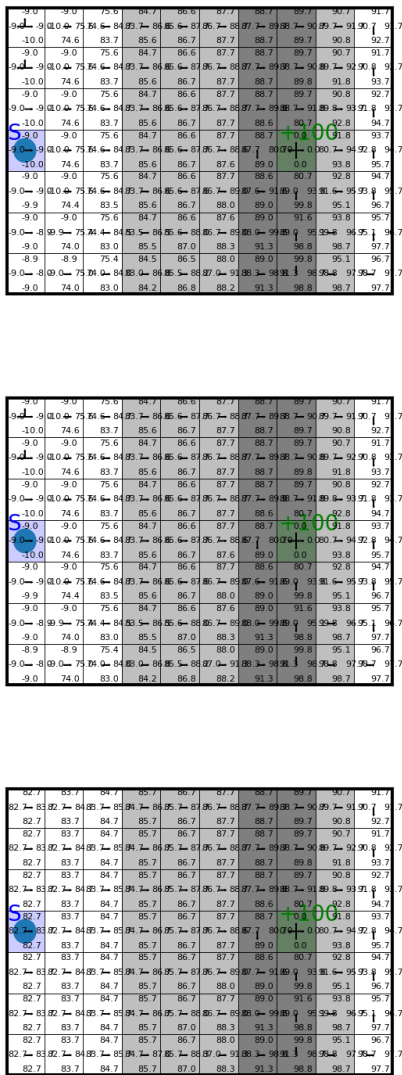


Figure 4. The changing Q captured when the code is initialized, at the midpoint and after convergence

C. Multi Goal Progression.

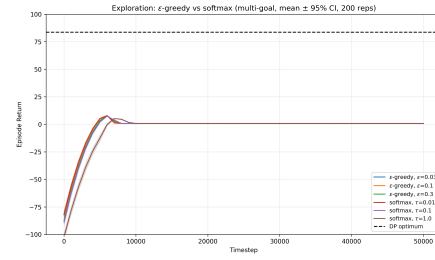


Figure 5. Learning curves with a secondary goal at (3,2) with $R = +5$, averaged over 200 runs with 95% confidence intervals. All strategies plateau near 0, exploiting the local goal. The DP optimum of ≈ 83.7 is never approached, highlighting how a nearby small reward can inhibit learning of a distant large reward.

D. Run to run variation.

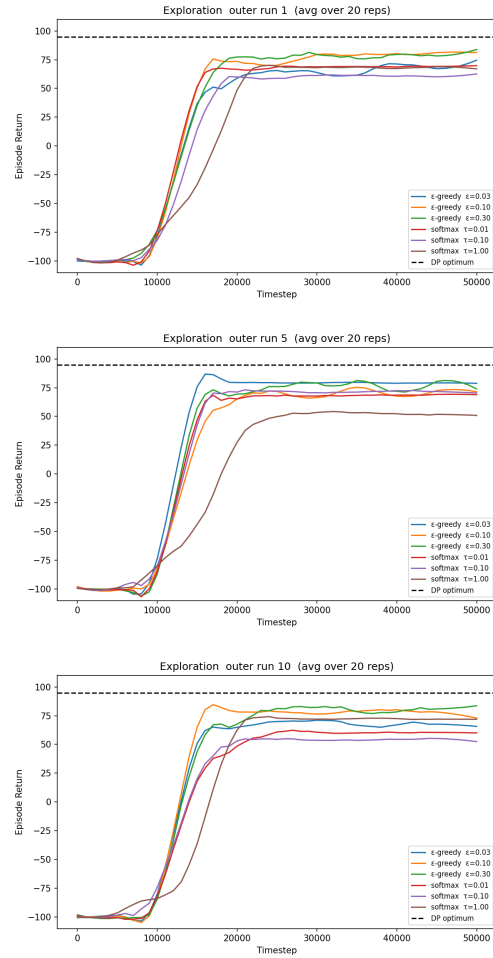


Figure 6. The Noticed run to run variation found in Exploration even when 20 repetitions are taken therefore motivating us to increase the repetitions to 200

Reinforcement Learning Assignment 1 - Tabular Reinforcement Learning

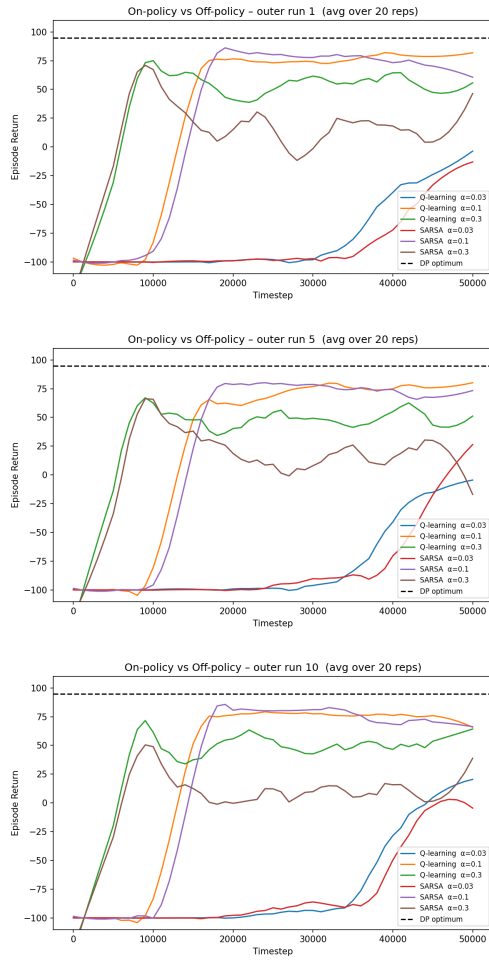


Figure 7. The Noticed run to run variation found in on/off policy even when 20 repetitions are taken therefore motivating us to increase the repetitions to 200

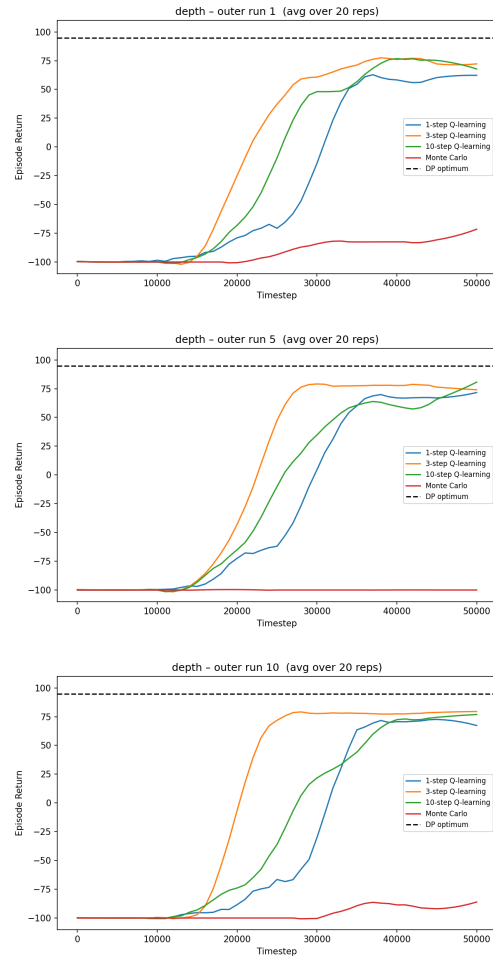


Figure 8. The Noticed run to run variation found in Depth even when 20 repetitions are taken therefore motivating us to increase the repetitions to 200